DS	**Session**	Stack	**Question Information**	Level 1 Challenge 45

Problem Description:

Sajid was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Sajid decided to book one ticket for out of those persons also along with his family members.

He wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, Check for Balanced Brackets in an expression using Stack

Can you help them to complete the task?

Function Description

- Declare a character stack S.
- Now traverse the expression string exp.
    1. If the current character is a starting bracket (**'('** or **'{'** or **'['**) then push it to stack.
    2. If the current character is a closing bracket (**')'** or **'}'** or **']'**) then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then "not balanced"

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
{()}[]
```

EXPECTED OUTPUT

```
Balanced
```

Test Case 2

INPUT (STDIN)

```
{({}])}
```

EXPECTED OUTPUT

```
Not Balanced
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
pop()
```

Test Case 2

KEYWORD

```
empty()
```

Test Case 3

KEYWORD

```
push
```

Test Case 4

KEYWORD

```
stack
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
15
```

Test Case 2

TOKEN COUNT

```
275
```

Test Case 3

NLOC

```
100
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

char stack[1000];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
int empty() { return top == -1; }

int isMatching(char a, char b) {
    if (a == '(' && b == ')') return 1;
    if (a == '{' && b == '}') return 1;
    if (a == '[' && b == ']') return 1;
    return 0;
}

int main() {
    char exp[1000];
    if (scanf("%s", exp) != 1) return 0;
    int balanced = 1;
    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];
        if (ch == '(' || ch == '{' || ch == '[') push(ch);
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (empty() || !isMatching(pop(), ch)) {
                balanced = 0;
                break;
            }
        }
    }
    if (balanced && empty()) printf("Balanced\n");
    else printf("Not Balanced\n");
    return 0;
}
```

Output

